

Системное программирование.

Лабораторная работа № 5

Задание на лабораторную работу

- 1) В программе необходимо реализовать функцию вычисления выражения на встроенном ассемблере с использованием сопроцессора.
- 2) Значения переменных передаются в качестве параметров функции.
- 3) Все параметры функции вещественные числа.
- 4) В качестве результата работы программы вывести на экран полученное значение.
- 5) Для проверки результата написать аналогичную функцию вычисления выражения на C++. Вывести результат на экран.
- 6) Подготовить тестовые примеры, приводящие к возникновению особых ситуаций.

Обязательным вопросом будет являться вопрос о представлении чисел в сопроцессоре.

Математический сопроцессор (FPU)

Важной частью микропроцессоров архитектуры Intel является наличие устройства для обработки числовых данных в формате с плавающей точкой. Сопроцессор дополняет основной процессор следующими новыми возможностями по обработке вещественного формата данных:

- полная поддержка стандартов IEEE-754 и -854 на арифметику с плавающей точкой. Эти стандарты описывают как форматы данных, с которыми должен работать сопроцессор, так и набор реализуемых им функций;
- поддержка численных алгоритмов для вычисления значений тригонометрических функций, логарифмов и т.п. без необходимости самостоятельной разработки соответствующих функций;
- обработка десятичных чисел с точностью до 18 разрядов, что позволяет сопроцессору выполнять арифметические операции без округления над целыми десятичными числами со значениями до 10^{18} ;
- обработка вещественных чисел из диапазона $3.37 \times 10^{-4932} \dots 1.18 \times 10^{+4932}$.

Набор регистров

С точки зрения программиста, сопроцессор представляет собой совокупность регистров, изображенную на рисунке 1, каждый из которых имеет свое функциональное назначение, набор типов данных и систему команд.

В программной модели сопроцессора можно выделить три группы регистров:

- восемь регистров $R0 - R7$, составляющих основу программной модели сопроцессора – стек сопроцессора;
- три служебных регистра: регистр состояния сопроцессора SWR , управляющий регистр сопроцессора CWR , регистр слова тегов TWR ;
- два регистра указателей: данных – DPR и команд – IPR .

Регистры стека сопроцессора

Размерность каждого из регистров $R0-R7$ 80 битов. Организация регистров в виде стека характерна для устройств, специализирующихся на обработке вычислительных алгоритмов и используют представление математического выражения с обратной польской записи (ПОЛИЗ). Вычисление такого выражения заключается в выборке из вершины стека очередной операции. Если это бинарная операция, то из вершины стека извлекаются два операнда, над которыми и производится действия в соответствии с выбранной ранее операцией. Реализация численных алгоритмов на основе регистрового стека позволяет получить существенный выигрыш в скорости вычислений.

Регистровый стек сопроцессора организован по принципу кольца. Это означает, что все регистры стека с функциональной точки зрения абсолютно одинаковы и равноправны. Как и в любом стеке, в стеке сопроцессора есть

вершина, которая является «плавающей». Контроль текущей вершины осуществляется аппаратно с помощью трехбитового поля *TOP* регистра *SWR*. В этом поле фиксируется физический номер (0...7) регистра стека, который в данный момент является текущей вершиной стека.

Команды сопроцессора не оперируют физическими номерами регистров стека *R0...R7*. Вместо этого они используют логические номера этих регистров *ST(0)...ST(7)*. С помощью логических номеров реализуется относительная адресация регистров стека сопроцессора. По мере записи в стек, указатель его вершины движется по направлению к младшим номерам физических регистров (уменьшается на единицу). Если текущей вершиной является *R0*, то после записи очередного значения в стек сопроцессора его текущей вершиной станет физический регистр *R7*. Таким образом, реализуется принцип кольца.



Рисунок 1 – Программная модель сопроцессора

Регистр управления CWR

Регистр управления работой сопроцессора определяет особенности обработки численных данных и состоит из:

- шести масок исключений;
- поля управления точностью *PC* (*Precision Control*);
- поля управления округлением *RC* (*Rounding Control*).

Шесть масок предназначены для маскирования исключительных ситуаций, возникновение которых фиксируется с помощью шести бит регистра *SWR*. Если какие-то биты исключений в регистре *CWR* установлены в единицу, то это означает, что соответствующие исключения будут обрабатываться самим сопроцессором. Если для какого-либо исключения в соответствующем бите масок исключений регистра *CWR* содержится нулевое значение, то при

возникновении исключения этого типа будет возбуждено прерывание 16 (10h). Операционная система должна содержать обработчик этого прерывания.

Поле управления точностью *PC* предназначено для выбора длины мантиссы. Возможные значения в этом поле означают:

00 – длина мантиссы 24 бита;

10 – длина мантиссы 53 бита;

11 – длина мантиссы 64 бита.

По умолчанию устанавливается значение поля $PC = 11$.

Поле управления округлением *RC* позволяет управлять процессом округления чисел в процессе работы сопроцессора. Необходимость операции округления может появиться в ситуации, когда после выполнения очередной команды сопроцессора получается не представимый результат, например, периодическая дробь 3,(3). Установив одно из значений в поле *RC*, можно выполнить округление в необходимую сторону. Например пусть *M* – значение в *ST(0)* или результат работы некоторой команды, который не может быть точно представлен и поэтому должен быть округлен, а числа *A* и *B* – наиболее близкие значения к значению *M*, которые могут быть представлены в регистре *ST(0)* сопроцессора, причем выполняется условие $A < M < B$, тогда возможные значения поля *RC* соответствуют следующим видам округления:

00 – значение *M* округляется к ближайшему числу *A* или *B*;

01 – значение *M* округляется в меньшую сторону, то есть $M = A$;

10 – значение *M* округляется в большую сторону, то есть $M = B$;

11 – производится отбрасывание дробной части *M*. Используется для приведения значения к форме, которая может использоваться в операциях целочисленной арифметики.

Регистр состояния *SWR*

Регистр *SWR* отражает текущее состояние сопроцессора после выполнения последней команды и состоит из:

- 6 флагов исключительных ситуаций;
- бита *SF* (*Stack Fault*) – ошибка работы стека сопроцессора. Бит устанавливается в единицу, если возникает одна из трех исключительных ситуаций (см. ниже) *PE*, *UE* или *IE*. В частности, его установка информирует о попытке записи в заполненный стек, или, напротив, попытке чтения из пустого стека. После анализа этого бита, его нужно снова установить в ноль, вместе с битами *PE*, *UE* или *IE* (если они были установлены);
- бита *ES* (*Error Summary*) – суммарная ошибка работы сопроцессора. Бит устанавливается в единицу, если возникает любая из шести перечисленных ниже исключительных ситуаций;
- четырех битов *C0-C3* (*Condition Code*) – кода условия. Назначение этих битов аналогично флагам в регистре *EFLAGS* основного

процессора – отразить результат выполнения последней команды сопроцессора. В описании команд для некоторых команд сопроцессора приведена интерпретация битов *C0-C3*;

- трехбитного поля *TOP*, которое содержит номер физического регистра являющегося текущей вершиной стека.

Первые 6 флагов регистра *SWR* предназначены для регистрации исключительных ситуаций. Все возможные исключения сведены к шести типам, каждому из которых соответствует один бит в регистре *SWR*. Сопроцессор самостоятельно реагирует на эти исключения, но для этого необходимо соответствующее исключение замаскировать. Такое действие выполняется с помощью установки в единицу нужного бита в управляющем регистре сопроцессора *CWR*. С помощью регистра *SWR* фиксируются следующие типы исключений:

- *IE (Invalid operation Error)* – недействительная операция;
- *DE (Denormalized operand Error)* – денормализованный операнд;
- *ZE (divide by Zero Error)* – ошибка деления на нуль;
- *OE (Overflow Error)* – ошибка переполнения. Возникает в случае выхода порядка числа за максимально допустимый диапазон;
- *UE (Underflow Error)* – ошибка антипереполнения. Возникает, когда результат слишком мал;
- *PE (Precision Error)* – ошибка точности. Устанавливается, когда сопроцессору приходится округлять результат из-за того, что его точное представление невозможно (например, сопроцессор никогда не сможет точно разделить 10 на 3).

При возникновении любого из этих шести типов исключений устанавливается в единицу соответствующий бит в регистре *SWR*, вне зависимости от того, было ли замаскировано это исключение в регистре *CWR* или нет.

Регистр тегов *TWR*

Регистр тегов *TWR* представляет собой совокупность двухбитовых полей. Каждое поле соответствует определенному физическому регистру стека и характеризует его текущее состояние. Изменение состояния любого из регистров стека сопроцессора отражается на содержимом соответствующего этому регистру поля регистра тега. Возможны следующие значения в полях регистра тега:

- 00 – регистр стека сопроцессора занят допустимым ненулевым значением;
- 01 – регистр стека сопроцессора содержит нулевое значение;
- 10 – регистр стека сопроцессора содержит одно из специальных численных значений (см. ниже), за исключением нуля;
- 11 – регистр пуст и в него можно производить запись (но это не означает, что все биты соответствующего регистра стека будут нулевыми).

Как уже отмечалось, при написании программы разработчик манипулирует не абсолютными, а относительными номерами регистров стека. По этой

причине могут возникнуть трудности при попытке интерпретации содержимого регистра тегов *TWR*, с соответствующими физическими регистрами стека. В качестве связующего звена необходимо привлекать информацию из поля *TOP* регистра *SWR*.

Регистры указатели *DPR* и *IPR*

Два регистра указателей – данных *DPR* (*Data Point Register*) и команд *IPR* (*Instruction Point Register*) предназначены для запоминания информации об адресе команды, вызвавшей исключительную ситуацию и адресе ее операнда. Эти указатели используются при обработке исключительных ситуаций (но не для всех команд).

2.6 Совместная работа процессора и сопроцессора

Процессор и сопроцессор имеют свои, несовместимые друг с другом системы команд и форматы обрабатываемых данных. Несмотря на то, что сопроцессор архитектурно представляет собой отдельное вычислительное устройство, он не может существовать отдельно от основного процессора. Первые модели процессоров Intel (*i8086*, *i286*, *i386*) и сопроцессоров (соответственно *i8087*, *i287*, *i387*) выполнялись как отдельные устройства – сопроцессор, при необходимости, вставлялся в специальный разъем на системной плате. Начиная с модели *i486*, сопроцессор и основной процессор производятся в одном корпусе и являются физически неделимыми, то есть архитектурно это по-прежнему два разных устройства, а аппаратно – одно. Процессор и сопроцессор являются двумя самостоятельными вычислительными устройствами и могут работать параллельно. Но этот параллелизм касается только их внутренней работы над исполнением очередной команды.

Оба устройства подключены к общей системной шине и имеют доступ к одинаковой информации. Основной процессор инициирует процесс выборки очередной команды, которая попадает одновременно в оба процессора. Любая команда сопроцессора имеет код операции, первые пять бит, которого имеют значение 11011. Когда код операции начинается этими битами, то основной процессор по дальнейшему содержимому кода операции выясняет, требует ли команда обращения к памяти. Если это так, то основной процессор формирует физический адрес операнда и обращается к памяти, после чего содержимое ячейки памяти выставляется на шину данных. Если обращение к памяти не требуется, то основной процессор заканчивает работу над данной командой (не делая попытки ее исполнения!) и приступает к декодированию следующей команды из текущего входного командного потока.

Что же касается сопроцессора, то выбранная команда, как уже было отмечено, попадает в него одновременно с основным процессором.

Сопроцессор, определив по первым пяти битам, что очередная команда принадлежит его системе команд, начинает ее исполнение. Если команда требовала операнд в памяти, то сопроцессор обращается к шине данных за чтением содержимого ячейки памяти, которое к этому моменту предоставлено основным процессором.

Из этой схемы взаимодействия следует, что в определенных случаях необходимо согласовывать работу обоих устройств. К примеру, если во входном потоке сразу за командой сопроцессора следует команда основного процессора, использующая результаты работы предыдущей команды, то сопроцессор не успеет выполнить свою команду за то время, когда основной процессор, пропустив сопроцессорную команду, выполнит свою. Очевидно, что логика работы программы будет нарушена.

Возможна и другая ситуация. Если входной поток команд содержит последовательность из нескольких команд сопроцессора, то, очевидно, что процессор в отличие от сопроцессора проскочит их очень быстро, чего он не должен делать, так как он обеспечивает внешний интерфейс для сопроцессора. Эти и другие, более сложные, ситуации приводят к необходимости синхронизации между собой работы двух процессоров. В первых моделях микропроцессоров это делалось путем вставки перед или после каждой команды сопроцессора специальной команды *wait* или *fwait*. Работа данной команды заключалась в приостановке работы основного процессора до тех пор, пока сопроцессор не закончит работу над последней командой. В моделях микропроцессора начиная с i486 подобная синхронизация выполняется командами *wait/fwait*, которые введены в алгоритм работы большинства команд сопроцессора. Но для некоторых команд (команды управления сопроцессором) оставлена возможность выбора между командами с синхронизацией и без нее.

Типы данных сопроцессора

3.1 Двоичные целые числа

Сопроцессор работает с тремя типами целых чисел, изображенными на рисунке 2.

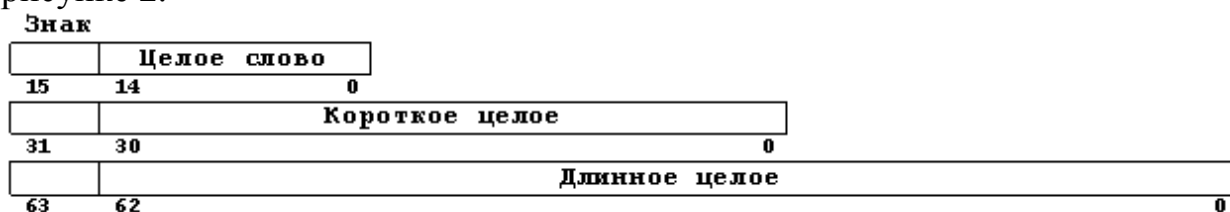


Рисунок 2 – Форматы целых чисел сопроцессора

В таблице 1 представлены формат целых чисел, их размерность и диапазон значений.

Таблица 1 – Форматы целых чисел сопроцессора

Формат	Размер, бит	Диапазон значений
Целое слово	16	-32 768...+32 767
Короткое слово	32	-2 147 483 648...+2 147 483 647
Длинное целое	64	-9 223 372 036 854 775 808...+9 223 372 036 854 775 807

Следует помнить, что сопроцессор преобразовывает целые числа в их внутреннее представление в виде эквивалентных вещественных чисел расширенного формата, поэтому работа с целыми числами осуществляется сопроцессором неэффективно (сопроцессор тратит время на преобразование).

3.3. Вещественные числа

Основной тип данных, с которым работает сопроцессор, вещественный. Данные этого типа описываются тремя форматами (рисунок 3): коротким, длинным и расширенным.

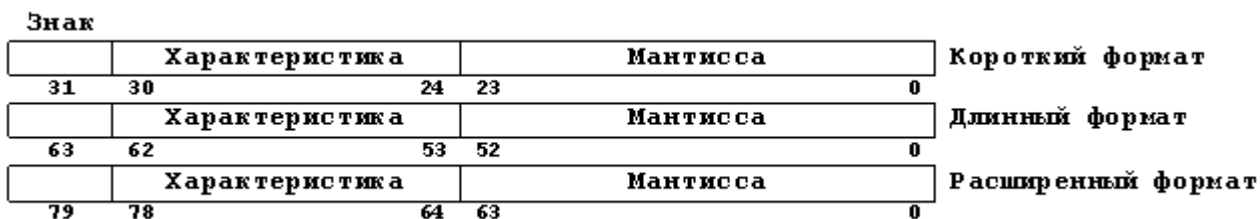


Рисунок 3 – Форматы вещественных чисел

В таблице 2 показаны диапазоны значений характеристик и мантисс вещественных чисел.

Таблица 2 – Форматы вещественных чисел

Формат	Короткий	Длинный	Расширенный
Длина числа (бит)	32	64	80
Размерность мантиссы (бит)	24	53	64
Размерность характеристики	8	11	15
Диапазон значений	$10^{-38} \dots 10^{+38}$	$10^{-308} \dots 10^{+308}$	$10^{-4932} \dots 10^{+4932}$

Специальные численные значения

Несмотря на большой диапазон вещественных значений, которые могут быть представлены в регистрах сопроцессора, есть бесконечное множество значений, которые находятся за рамками этого диапазона. Для того чтобы иметь возможность реагировать на вычислительные ситуации, в которых возникают такие значения, в сопроцессоре предусмотрены специальные комбинации бит, называемые специальными численными значениями.

Денормализованные вещественные числа – это числа, которые меньше минимального положительного нормализованного числа для каждого вещественного формата (рисунок 4).

Сопроцессор хранит числа в нормализованном виде. По мере приближения чисел к нулю сопроцессору все труднее «вытягивать» их значения к нормализованному виду, то есть к такому виду, чтобы первой значащей цифрой мантиссы была единица.

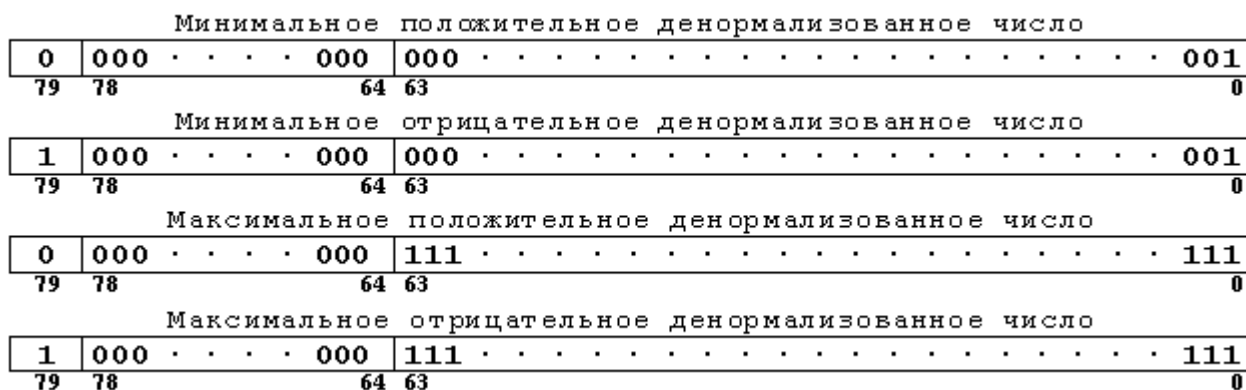


Рисунок 4 – Диапазон денормализованных чисел

Нуль. Значение нуля также относится к специальным численным значениям, это делается из-за того, что это значение особо выделяется среди корректных вещественных значений, формируемых как результат работы некоторой команды.

Значение истинного нуля может иметь знак (рисунок 5), что, впрочем, не влияет на его использование командами сопроцессора.

вычислений рекомендуется периодически контролировать результаты исполнения команд на предмет появления спокойных нечисел.

Система команд сопроцессора

Команды передачи данных

Группа команд передачи данных предназначена для организации обмена между регистрами стека, вершиной стека сопроцессора и ячейками оперативной памяти. С помощью этих команд осуществляются все перемещения значений операндов в сопроцессор и из него.

В сопроцессоре имеются следующие команды передачи данных:

fld/fild источник – загрузка вещественного/целого числа из памяти в вершину стека сопроцессора;

fst/fist приемник – сохранение вещественного/целого числа из вершины стека сопроцессора в память. Сохранение числа в памяти не сопровождается выталкиванием его из стека, то есть текущая вершина стека сопроцессора не изменяется (поле *top* не изменяется);

fstp/fistp приемник – сохранение вещественного/целого числа из вершины стека сопроцессора в память. В отличие от предыдущей команды происходит выталкивание вещественного числа из стека после его сохранения в памяти. Команда изменяет поле *top*, увеличивая его на единицу. Вследствие этого вершиной стека становится следующий, больший по своему физическому номеру, регистр стека сопроцессора;

fxch st(i) – обмен значений между текущей вершиной стека и регистром стека сопроцессора *ST(i)*.

Пример:

```
double x = 2.7, y = 1.5;
int a = 67, b;
__asm {
    fld      x      ;st(0) = x
    fld      y      ;st(0) = y, st(1) = x
    fstp     x      ;x = st(0), st(0) = x
    fst      y      ;y = st(0)
    fistp    b      ;c = st(0) (с округлением)
    fild     a      ;st(0) = a
}
```

Команды загрузки констант

В математических вычислениях достаточно часто встречаются предопределенные константы. Сопроцессор хранит значения некоторых из них. Для каждой предопределенной константы существует своя специальная команда:

fldz – загрузка нуля в вершину стека сопроцессора;

fldl – загрузка единицы в вершину стека сопроцессора;

fldpi – загрузка числа π в вершину стека сопроцессора;

fldl2t – загрузка двоичного логарифма десяти ($\log_2 10$) в вершину стека сопроцессора;

fldl2e – загрузка двоичного логарифма e ($\log_2 e$) в вершину стека сопроцессора;

fldlg2 – загрузка десятичного логарифма двух ($\lg 2$) в вершину стека сопроцессора;

fldln2 – загрузка натурального логарифма двух ($\ln 2$) в вершину стека сопроцессора.

Команды сравнения данных

Команды данной группы выполняют сравнение значений числа, находящегося в вершине стека и операнда, указанного в команде:

fcom/ficom [операнд] – команда без операндов сравнивает два значения: одно находится в регистре $ST(0)$, другое – в регистре $ST(1)$. Если указан *операнд*, то сравнивается значение в регистре $ST(0)$ стека сопроцессора со значением в памяти. По результатам сравнения устанавливаются биты $C3$, $C2$, $C0$ регистра SWR ;

fcomp/ficomp операнд – команда сравнивает значение в вершине стека сопроцессора $ST(0)$ со значением операнда, который находится в регистре или в памяти. После сравнения и установки бит $C3$, $C2$, $C0$ команда выталкивает значение из $ST(0)$. Длина целого операнда 16 или 32 бита, то есть целое слово и короткое целое;

ftst – команда не имеет операндов и сравнивает значение в $ST(0)$ со значением 0.

В результате работы приведенных команд сравнения в регистре состояния SWR устанавливаются биты $C3$, $C2$, $C0$ как указано в таблице 3:

Таблица 3 – Результаты выполнения сравнения

Результат операции	C3	C2	C0
операнды несравнимы	1	1	1
<i>операнд 2</i> > <i>операнд 1</i> ($ST(1) > ST(0)$)	0	0	1
<i>операнд 2</i> < <i>операнд 1</i> ($ST(1) < ST(0)$)	0	0	0
<i>операнд 2</i> = <i>операнд 1</i> ($ST(1) = ST(0)$)	1	0	0

Для программирования реакции на результаты сравнения необходимо анализировать состояние этих битов. Сопроцессор не имеет для этого

специальных команд, за исключением команды *fstsw*, которая позволяет записать содержимое регистра *SWR* в регистр *AX*. Затем командой *sahf* содержимое регистра *AH*, в котором находятся биты *C3*, *C2* и *C0*, записывается в младший байт регистра *EFLAGS/FLAGS*. Местоположение бит *C3*, *C2* и *C0* соответствует местоположению флагов *ZF*, *PF* и *CF*. Таким образом, после применения команды *sahf* становится возможным реагировать на результаты сравнения значений в сопроцессоре командами условного перехода основного процессора.

Например, приведенная ниже функция возвратит 0 если *a* и *b* несравнимы, 1 если *a* < *b*, 2 если *a* > *b* и 3 если *a* = *b*.

```
int compare_real(double a, double b)
{
    int r = 0;
    __asm {
        fld        a
        fld        b
        fcom
        fstsw      ax
        sahf                ;cf = c3, pf = c2, zf = c0
        jp         not_cmp  ;если числа несравнимы
        jc         if_less  ;если a < b
        jz         if_equal ;если a = b
        jmp        if_great ;если a > b
    not_cmp:
        xor        eax, eax ;вернуть 0
        jmp        end_cmp
    if_less:
        mov        eax, 1   ;вернуть 1
        jmp        end_cmp
    if_great:
        mov        eax, 2   ;вернуть 2
        jmp        end_cmp
    if_equal:
        mov        eax, 3   ;вернуть 3
    end_cmp:
        mov        r, eax
    }
    return r;
}
```

Арифметические команды

Команды сопроцессора, входящие в данную группу, реализуют четыре основные арифметические операции – сложение, вычитание, умножение и деление. Имеется также несколько дополнительных команд, предназначенных для повышения эффективности использования основных арифметических команд.

Целочисленные арифметические команды:

fiadd источник – команда складывает значения $ST(0)$ и целочисленного источника, в качестве которого выступает 16- или 32-разрядный операнд в памяти. Результат сложения запоминается в регистре стека сопроцессора $ST(0)$;

fisub источник – команда вычитает значение целочисленного источника из $ST(0)$. Результат вычитания запоминается в регистре стека сопроцессора $ST(0)$. В качестве источника выступает 16- или 32-разрядный целочисленный операнд в памяти;

fi mul источник – команда умножает значение целочисленного источника на содержимое $ST(0)$. Результат умножения запоминается в регистре стека сопроцессора $ST(0)$. В качестве источника выступает 16- или 32-разрядный целочисленный операнд в памяти;

fidiv источник – команда делит содержимое $ST(0)$ на значение целочисленного источника. Результат деления запоминается в регистре стека сопроцессора $ST(0)$. В качестве источника выступает 16- или 32-разрядный целочисленный операнд в памяти.

Вещественные арифметические команды:

fadd – команда складывает значения из регистров $ST(0)$ и $ST(1)$. Результат сложения запоминается в регистре стека сопроцессора $ST(0)$;

fadd слагаемое – команда складывает значения $ST(0)$ и *слагаемого*, представляющего собой адрес ячейки памяти. Результат сложения запоминается в регистре стека сопроцессора $ST(0)$;

fadd st(i), st – команда складывает значение в регистре стека сопроцессора $ST(i)$ со значением в вершине стека $ST(0)$. Результат сложения запоминается в регистре $ST(i)$;

faddp st(i), st – команда производит сложение вещественных операндов аналогично команде *fadd st(i), st*. Последним действием команды является выталкивание значения из вершины стека сопроцессора $ST(0)$. Результат сложения остается в регистре $ST(i-1)$;

fsub – команда вычитает из значения в регистре $ST(1)$ значение регистра $ST(0)$. Результат вычитания запоминается в регистре стека сопроцессора $ST(0)$;

fsub вычитаемое – команда вычитает значение *вычитаемого* из значения в $ST(0)$. *Вычитаемое* представляет собой адрес ячейки памяти, содержащей допустимое вещественное число. Результат сложения запоминается в регистре стека сопроцессора $ST(0)$;

fsub st(i), st – команда вычитает значение в вершине стека $ST(0)$ из значения в регистре стека сопроцессора $ST(i)$. Результат вычитания запоминается в регистре стека сопроцессора $ST(i)$;

fsubp st(i), st – команда вычитает вещественные операнды аналогично команде *fsubp st(i), st*. Последним действием команды является выталкивание значения из вершины стека сопроцессора $ST(0)$. Результат вычитания остается в регистре $ST(i-1)$;

fmul – команда (без операндов) умножает значение в $ST(1)$ на содержимое регистра в $ST(0)$. Результат умножения запоминается в регистре стека сопроцессора $ST(0)$;

fmul множитель – команда умножает *множитель* на содержимое регистра стека $ST(0)$. Результат умножения запоминается в регистре стека сопроцессора $ST(0)$;

fmul st(i), st – команда умножает значение в произвольном регистре $ST(i)$ на содержимое регистра стека $ST(0)$. Результат умножения запоминается в регистре стека сопроцессора $ST(i)$;

fmulp st(i), st – команда производит умножение подобно команде *fmul st(i), st*. Последним действием команды является выталкивание значения из вершины стека сопроцессора $ST(0)$. Результат умножения остается в регистре $ST(i-1)$;

fdiv – команда (без операндов) делит содержимое регистра $ST(1)$ на значение регистра сопроцессора $ST(0)$. Результат деления запоминается в регистре стека сопроцессора $ST(0)$;

fdiv делитель – команда делит содержимое регистра $ST(0)$ на *делитель*. Результат деления запоминается в регистре стека сопроцессора $ST(0)$;

fdiv st(i), st – команда производит деление содержимого регистра $ST(i)$ на значение регистра сопроцессора $ST(0)$. Результат деления запоминается в регистре стека сопроцессора $ST(i)$;

fdivp st(i), st – команда производит деление аналогично команде *fdiv st(i), st*. Последним действием команды является выталкивание значения из вершины стека сопроцессора $ST(0)$. Результат деления остается в регистре $ST(i-1)$.

Дополнительные арифметические команды:

fsqrt – вычисление квадратного корня из значения, находящегося в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов. Результат вычисления помещается в регистр *ST(0)*. Следует отметить, что данная команда обладает определенными достоинствами. Во-первых, результат извлечения достаточно точен, во-вторых, скорость исполнения чуть больше скорости выполнения команды деления вещественных чисел *fdiv*;

fabs – вычисление модуля значения, находящегося в вершине стека сопроцессора – регистре *ST(0)*. Команда не имеет операндов. Результат вычисления помещается в регистр *ST(0)*;

fchs – изменение знака значения, находящегося в вершине стека сопроцессора – регистре *ST(0)*. Команда не имеет операндов. Результат вычисления помещается обратно в регистр *ST(0)*. Отличие команды *fchs* от команды *fabs* в том, что команда *fchs* только инвертирует знаковый разряд значения в регистре *ST(0)*, не меняя значения остальных бит. Команда вычисления модуля *fabs* при наличии отрицательного значения в регистре *ST(0)*, наряду с инвертированием знакового ряда, выполняет изменение остальных бит значения таким образом, чтобы в *ST(0)* получилось соответствующее положительное число;

fextract – команда выделения порядка и мантиссы значения, находящегося в вершине стека сопроцессора – регистре *ST(0)*. Команда не имеет операндов. Результат выделения помещается в два регистра стека – мантисса в *ST(0)*, а порядок в *ST(1)*. При этом мантисса представляется вещественным числом с тем же знаком, что и у исходного числа, и порядком равным нулю. Порядок, помещенный в *ST(1)*, представляется как истинный порядок, то есть без константы смещения, в виде вещественного числа со знаком и соответствует величине p формулы $A=(\pm M)*N^{\pm(p)}$;

fscale – команда умножает или делит значение, находящееся в вершине стека сопроцессора *ST(0)*, на степень двух. Значение показателя степени находится в регистре стека *ST(1)* и представляет собой целое число со знаком. Если показатель степени не является целым числом, то он округляется до целого в меньшую сторону. Команда не имеет операндов. Команда *fscale* не очищает регистр *ST(1)*. Результат помещается в регистр *ST(0)*;

frndint – команда округления до целого значения – округляет значение, находящееся в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов.

Команды трансцендентных функций

Сопроцессор имеет ряд команд, предназначенных для вычисления значений тригонометрических функций, таких, как синус, косинус, тангенс, арктангенс, а так-же значений логарифмических и показательных функций. Наличие этих команд значительно облегчает разработку вычислительных алгоритмов. В системе команд сопроцессора имеются следующие трансцендентные команды:

fcos – команда вычисляет косинус угла в радианах, находящегося в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов. Результат возвращается в регистр *ST(0)*;

fsin – команда вычисляет синус угла в радианах, находящегося в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов. Результат возвращается в регистр *ST(0)*;

fsincos – команда вычисляет синус и косинус угла в радианах, находящегося в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов. Результат возвращается в регистры *ST(0)* и *ST(1)*. При этом косинус помещается в *ST(0)*, а синус в *ST(1)*;

fptan – команда вычисляет частичный тангенс угла в радианах, (в диапазоне $-2^{63} \dots 2^{63}$), находящегося в вершине стека сопроцессора *ST(0)*. Команда не имеет операндов. Результат возвращается в регистры *ST(0)* – значение единицы, и *ST(1)* – тангенс угла;

fpatan – команда вычисляет частичный арктангенс частного x/y . (x находится в регистре *ST(0)*, y – в регистре *ST(1)*). Команда не имеет операндов. Результат возвращается в регистре *ST(0)*;

fprem – команда получения частичного остатка от деления. Исходные значения делимого и делителя размещаются в стеке – делимое в *ST(0)*, делитель в *ST(1)*. Делитель рассматривается как некоторый модуль, поэтому в результате работы команды получается остаток от деления по модулю. Но произойти это может не сразу, так как этот результат в общем случае достигается за несколько производимых подряд обращений к команде *fprem*. Это происходит, если значения операндов сильно различаются. Физическая работа команды заключается в реализации хорошо известного действия деления в столбик. При этом каждое промежуточное деление осуществляется отдельной командой *fprem*. Цикл, центральное место в котором занимает команда *fprem*, завершается, когда очередная полученная разность в *ST(0)* становится меньше значения модуля в *ST(1)*. Судить об этом можно по состоянию флага *C2* в регистре состояния *SWR*:

если $C2 = 0$, то работа команды *fprem* полностью завершена, так как разность *ST(0)* меньше значения модуля в *ST(1)*;

если $C2 = 1$, то необходимо продолжить выполнение команды *fprem*, так как разность *ST(0)* больше значения модуля в *ST(1)*;

f2xm1 – команда вычисления значения функции $y = 2^x - 1$. Исходное значение x размещается в вершине стека сопроцессора в регистре *ST(0)* и

должно лежать в диапазоне $-1 \leq x \leq 1$. Результат y замещает x в регистре $ST(0)$. Эта команда может быть использована для вычисления различных показательных функций;

$fyl2x$ – команда вычисления значения функции $z = y \log_2(x)$. Исходное значение x размещается в вершине стека сопроцессора, а исходное значение y – в регистре $ST(1)$. Значение x должно лежать в диапазоне $0 \leq x \leq +\infty$, а значение y – в диапазоне $-\infty \leq y \leq +\infty$. Перед тем, как осуществить запись результата z в вершину стека, команда $fyl2x$ выталкивает значения x и y из стека и только после этого производит запись z в регистр $ST(0)$;

$fyl2xp1$ – команда вычисления значения функции $z = y \log_2(x+1)$. Исходное значение x размещается в вершине стека сопроцессора – регистре $ST(0)$, а исходное значение y – в регистре $ST(1)$.). Значение x должно лежать в диапазоне $0 \leq x \leq 1-1/\sqrt{2}$, а значение y – в диапазоне $-\infty \leq y \leq +\infty$. Перед тем, как осуществить запись результата z в вершину стека, команда $fyl2xp1$ выталкивает значения x и y из стека и только после этого производит запись z в регистр $ST(0)$.

Пример. Необходимо вычислить значение следующего выражения $z = x \log_4(y) \operatorname{ctg}(y+x)$ при $x = 1.32$ и $y = 4.19$.

```
double x = 1.32, y = 4.19, z, ol = 4.0;
__asm {
    finit
    fldl      ;
    fld      ol      ;
    fyl2x          ;st(0) = 1*log2(4) = t1
    fld      x      ;
    fld      y      ;
    fyl2x          ;st(0) = x*log2(y) = t2, st(1) = t1
    fdiv          ;st(0) = t2/t1 = t3
    fld      x      ;
    fld      y      ;
    fadd          ;st(0) = x+y = t4, st(1) = t3
    fptan          ;st(0) = 1, st(1) = tg(t4), st(2) = t3
    fdiv          ;st(0) = 1/tg(t4) = t5, st(1) = t3
    fmul          ;st(0) = t5*t3
    fstp      z      ;z = x*log4(y)*ctg(x+y)
}
printf("z = %lf\n", z);
```

Команды управления сопроцессором

Последняя группа команд предназначена для общего управления работой сопроцессора:

wait/fwait – команда ожидания, предназначена для синхронизации работы процессора и сопроцессора;

finit – команда инициализации сопроцессора. Данную команду используют перед первой командой сопроцессора в программе и в других случаях, когда необходимо привести сопроцессор в начальное состояние;

fstsw назначение – команда сохранения содержимого регистра состояния *SWR* в ячейке памяти размером в 2 байта или регистре *AX*;

fstcw приемник – команда сохраняет содержимое регистра управления сопроцессора в 16-битную ячейку памяти, адресуемую операндом *приемник*. Команда позволяет получить доступ к части информации рабочей среды сопроцессора, а именно, к содержимому регистра *CWR*. В зависимости от особенностей конкретного алгоритма интерес представляют все поля регистра *CWR* – маски исключений, управление точностью и округлением. После их анализа можно установить новые значения полей и с помощью команды *fldcw* записать обновлённое слово обратно в регистр *CWR*;

fldcw источник – команда выполняет операцию загрузки значения регистра *CWR* из 16-битной ячейки памяти, адресуемой операндом *источник*. Это позволяет изменить определённые режимы работы сопроцессора, например, размаскировать нужные исключения;

ffree st(i) – команда освобождения регистра стека *ST(i)*. Команда записывает в поле регистра тегов, соответствующего регистру *ST(i)*, значение *11b*, что соответствует пустому регистру.

Пример преобразования вещественного числа в целое путем отбрасывания дробной части:

```
double d = 123.456;
int x;
short cw;
__asm {
    finit
    fstcw    cw
    or      cw, 0000110000000000b
    fldcw   cw
    fld     d
    fist    x
    finit
}
```

Пример вычисления выражения $(b + a*a - 1) / (25/c - b + 2);$

```
{  
  
    const double  
        c1=1,  
        c25=25,  
        c2=2;  
__asm{  
    finit  
    fld     qword ptr [a]  
    fmul    qword ptr [a]  
    fadd    qword ptr [b]  
    fsub    qword ptr [c1]  
    fld     qword ptr [c25]  
    fld     qword ptr [c]  
    fldz  
    fucompp  
    fnstsw ax  
    sahf  
    jz      error_0  
    fdiv    qword ptr [c]  
    fsub    qword ptr [b]  
    fadd    qword ptr [c2]  
    fldz  
    fucomp  
    fnstsw ax  
    sahf  
    jz      error_0  
    fdivp   st(1),st  
    fxam  
  
error_0:  
    }  
}
```

--	--	--	--	--

Результат из функции возвращаем в вершине сопроцессора.

Показатели работы:

Показатель	Сложность	Пояснение
Точность	2	Реализованы необходимые проверки
Оптимальность	1	Отсутствие «лишних» действий
Полнота	1	Комментарии о состоянии регистров сопроцессора
Вопрос по коду	2	
Обязательный вопрос	2	
Контрольный вопрос	3	
Бонус/Штраф	1	

Контрольные вопросы к лабораторной работе № 5

1. Регистры стека сопроцессора
2. Регистр управления
3. Регистр состояния
4. Типы данных сопроцессора
5. Команды передачи данных
6. Команды загрузки констант
7. Команды сравнения данных
8. Арифметические команды сложения и вычитания
9. Арифметические команды умножения и деления
10. Дополнительные арифметические команды:
11. Команды трансцендентных функций
12. Команды управления сопроцессором